RAG Chatbot + Evaluation — 4-Day Build Plan

(includes retrieval strategy comparison + chunking ablation)

Goal: A working RAG Q&A system over a document set, served via FastAPI, with a real evaluation pipeline (retrieval + faithfulness metrics), a retrieval-strategy comparison (dense vs hybrid vs reranked), and a chunking ablation study — genuinely defensible in an interview.

Tech Stack

- **LangChain** — orchestration
- **ChromaDB** — vector store (local, no setup needed)
- **rank_bm25** — sparse/keyword retrieval (for hybrid search)
- **sentence-transformers (cross-encoder)** — reranking
- **OpenAI / Groq / local (Ollama)** — embeddings + LLM (pick based on budget — Groq has a free tier and is fast)
- **FastAPI** — serving
- **RAGAS** — evaluation framework
- **Docker** (optional, but you already know this from Sentinel AI — reuse it to look consistent across projects)

Folder Structure

```
rag-chatbot/
├── data/                    # source PDFs/docs
├── ingest.py                # chunk + embed + store
├── rag_pipeline.py          # retrieval + generation logic
(dense/hybrid/rerank modes)
├── app.py                   # FastAPI server
├── eval/
│   ├── eval_questions.json      # your 20-30 Q&A pairs
│   ├── evaluate.py              # RAGAS scoring script
│   ├── compare_retrievers.py    # dense vs hybrid vs rerank comparison
│   └── chunking_ablation.py     # chunk size sweep + RAGAS re-scoring
├── results/
│   ├── retriever_comparison.csv
│   └── chunking_ablation.csv
├── requirements.txt
├── Dockerfile
└── README.md
```

Day 1 — Ingestion + Retrieval Pipeline

Step 1: Pick your document set

Choose ONE focused domain — don't mix topics. Good options:

- Your own college course notes/PDFs (10-20 files)
- A specific open-source library's documentation (e.g., FastAPI docs, LangChain docs)
- A set of 10-15 research papers on one topic

Smaller and focused = better retrieval quality = better eval scores = better resume claim.

Step 2: Install dependencies

```bash
pip install langchain langchain-community langchain-openai chromadb pypdf fasta
```

Step 3: Ingestion script (`ingest.py`)

- Load PDFs with `PyPDFLoader` or `DirectoryLoader`
- Split with `RecursiveCharacterTextSplitter` (chunk_size=500-800, overlap=100)
- Embed with `OpenAIEmbeddings` or a free alternative (`sentence-transformers/all-MiniLM-L6-v2` via HuggingFace if you want zero API cost)
- Store in ChromaDB with `persist_directory`

Step 4: Retrieval + generation (`rag_pipeline.py`)

- Load the persisted Chroma store
- Build a retriever (`k=4` similarity search)
- Wire up a `RetrievalQA` chain or manual prompt template: retrieved context + question → LLM answer
- Test manually with 5-10 sample questions in a notebook/script

End of Day 1 checkpoint: you can ask a question in a Python script and get a grounded answer back.

Day 2 — API + Evaluation

Step 5: FastAPI wrapper (`app.py`)

Minimum endpoints:

- `POST /query` — takes a question, returns answer + source chunks
- `GET /health` — returns status, model name, number of indexed chunks

This mirrors the Sentinel AI FastAPI pattern you already have — reuse that structure so both projects look like coherent engineering work.

Step 6: Build your eval set (`eval/eval_questions.json`)

Write 20-30 question/expected-answer pairs **based on your actual documents**. This is the part most people skip — don't skip it, it's what makes the project defensible.

```json
[
  {"question": "...", "ground_truth": "..."},
  ...
]
```

Step 7: Run RAGAS evaluation (`eval/evaluate.py`)

RAGAS gives you these metrics automatically:

- **Faithfulness** — is the answer actually grounded in retrieved context (hallucination check)
- **Answer relevance** — does the answer address the question
- **Context precision/recall** — is retrieval actually pulling the right chunks

```python
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, co
# run your pipeline over eval_questions.json, collect answers + contexts
# then: evaluate(dataset, metrics=[faithfulness, answer_relevancy, context_prec
```

This prints real numbers — this is your baseline metric, used for comparison in Day 3-4.

Day 3 — Retrieval Strategy Comparison (Dense vs Hybrid vs Reranked)

Step 9: Add a sparse retriever (BM25)

- Use `rank_bm25`'s `BM25Retriever` (LangChain has a wrapper: `langchain.retrievers.BM25Retriever`) over the same chunked documents
- This gives you keyword-based retrieval to compare against your existing dense (embedding) retrieval

Step 10: Build a hybrid retriever

- Use LangChain's `EnsembleRetriever` to combine dense + BM25 (e.g., weights `[0.5, 0.5]`)
- This is a few lines of config, not new infrastructure — reuses what you already built

Step 11: Add a reranker

- Load a cross-encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2` via `sentence-transformers`)
- After hybrid retrieval pulls top-10 candidates, rerank them and keep top-4 by cross-encoder score
- LangChain's `ContextualCompressionRetriever` + `CrossEncoderReranker` wraps this cleanly

Step 12: Run comparison (`eval/compare_retrievers.py`)

- Re-run your eval_questions.json through all 3 retrieval modes: dense-only, hybrid, hybrid+rerank
- Score each with the same RAGAS metrics from Step 7
- Save results to `results/retriever_comparison.csv`

End of Day 3 checkpoint: a table showing faithfulness/precision/recall for 3 retrieval strategies — you can now say which one wins and why.

Day 4 — Chunking Ablation Study

Step 13: Parametrize your ingestion

- Modify `ingest.py` so chunk_size and chunk_overlap are function arguments, not hardcoded
- Re-run ingestion for 3-4 chunk sizes (e.g., 300, 500, 800, 1200 tokens), each into its own Chroma collection so they don't overwrite each other

Step 14: Run the ablation (`eval/chunking_ablation.py`)

- For each chunk size, run your **best retrieval strategy from Day 3** (likely hybrid+rerank) over the eval set
- Score with RAGAS, save to `results/chunking_ablation.csv`

Step 15: Pick your final configuration

- Compare all results, pick the best chunk size + retrieval strategy combination
- Rebuild your final production index with this configuration — this is what your `app.py` should serve by default

End of Day 4 checkpoint: two CSVs of real experimental data, and a documented, justified final configuration — this is what separates this project from a tutorial copy.

Optional: Dockerize (if time permits)

Reuse your Sentinel AI Dockerfile pattern — same FastAPI + Docker Compose structure. Consistency across projects signals real engineering habits, not one-off scripts.

What Your Resume Bullets Should Look Like (fill in real numbers after eval runs)

```
RAG-Based Document Q&A System — GitHub | LangChain, ChromaDB, FastAPI, RAGAS
- Built a retrieval-augmented Q&A pipeline over [N] documents using LangChain
  and ChromaDB, served via a FastAPI endpoint with source-attributed
responses.
- Compared dense, hybrid (BM25 + dense), and reranked retrieval strategies on
a
  20-30 question benchmark, improving context precision by [X]% via hybrid
  retrieval with cross-encoder reranking.
- Ran a chunking ablation study across [N] chunk sizes, selecting the optimal
  configuration ([size] tokens) based on RAGAS faithfulness and precision
scores.
```

Guardrails (so this survives an interview)

- Don't claim numbers before you've actually run the eval scripts and seen them.
- Be ready to explain: what happens on a question with no good answer in the docs (does it hallucinate or say "I don't know"?). If you haven't handled this, add a simple prompt instruction: "If the context doesn't contain the answer, say you don't know" — this alone is a good interview talking point.
- Know your chunk size/overlap choices and *why* — you'll now have actual data to back this up instead of guessing.
- Be ready to explain why hybrid/reranking helped (or didn't) — e.g., BM25 catches exact keyword matches (names, IDs) that dense embeddings sometimes miss.

Time Budget Check

- Day 1: ~4-6 hrs (ingestion + retrieval, most of the debugging time goes here)
- Day 2: ~4-6 hrs (API wrapper is quick, eval script + running RAGAS takes the most time)
- Day 3: ~4-6 hrs (BM25 + ensemble retriever are quick to wire up; reranker integration and re-running eval across 3 modes takes longer)

- Day 4: ~3-5 hrs (mostly re-running the pipeline across chunk sizes — mechanical once Day 1-3 code is stable)

If you're tight on time, **cut the Docker step first**, then consider dropping reranking (Step 11) and just comparing dense vs hybrid — the eval numbers matter more for the resume claim than having all three modes.